Generisk kod …

… med typparametrar, <T>

En icke-generisk list-klass, för en specifik typ

- Vi kan hittills skriva t.ex. listklasser för en specifik typ …
- … men vi behöver skriva en ny listklass om vi vill hantera en annan typ …

```
public class ListOfStrings {
   private String[] theList;
   private int noOfElements;
    …

    public ListOfStrings() {
         theList = new String[10];
         noOfElements = 0;
    }

    public void addString(String s) { …
    public String getString(int index) { …
```

En list-klass för flera typer – före generics

- Supertypen Object, får referera till objekt av alla (sub-) typer …
- … så vi kan skriva en listklass som kan hantera alla typer av objekt …

```
public class ListOfObjects {
    private Object[] theList;
    private int noOfElements;
    …

    public ListOfObjects() {
        theList = new Object[10];
        noOfElements = 0;
    }

    public void addObject(Object o) { …
    public Object getObject(int index) { …
```

En list-klass för flera typer – före generics

- Skapa en instans av listan med Object-referenser

```
ListOfObjects list = new ListOfObjects();
list.addObject(new String("Forty-two"));
list.addObject(new Integer(42));
```

- Accessa objekt i listan

```
String s = (String) list.getObject(0);
Integer i = (Integer) list.getObject(1);
```

- ”Down cast” av referensens typ till aktuell typ nödvändig
- Är det refererade objektet av rätt typ? *Kontrolleras först run-time!*

En list-klass *med* Generics

- Vi vill kunna skriva klasser där typen på en datamedlem inte anges förrän instanser/objekt av klassen skapas, s.k. generisk kod
- Finns redan sådana klasser i Javas API, t.ex. java.util.ArrayList

```
ArrayList<String> strList = new ArrayList<String>();

strList.add(new String("Forty-two"));

String s = strList.get(0);
```

- <String> anger vilken typ listan ska instansieras för, en s.k. typparameter
- Kompilatorn kan kontrollera att rätt typ används. Följande ger kompileringsfel:
  `strList.add(new Integer(42));`

Generics, syntax för en enkel klass

- En generisk klass (mall) för att lagra data av godtycklig typ - T är en s.k. typparameter

```
public class DataSample<T> {

   private T theData;

   public DataSample(T theData) {
        this.theData = theData;
   }

   public T getData() {
        return theData;
   }

   public String toString() {
        return theData.toString();
   }
}
```

Generics, syntax för klass

- När man skapar instanser, objekt, av denna typ anger man vilken typ T ska ersättas med för just detta objekt

```
DataSample<String> dss = new DataSample<String>("Forty-two");
String s = dss.getData();


DataSample<Integer> dsi = new DataSample<Integer>(42);
int i = dsi.getData();
```

Generics, typparameter för enskild metod

- Generisk metod, typparametern endast giltig för metoden

```
public static <T> T getMiddleElement(T[] arr) {
   return arr[arr.length/2];
}
```

- Anrop

```
String[] words = {"C#", "Android", "Java ME"};
   …
String middle =
   Generics.<String>getMiddleElement(words);
```

En generisk list-klass

```
public class GenericList<E> {

    private E[] elements;
    private int n;

    public GenericList(int initialCapacity) {
        this.elements =
            (E[]) new Object[initialCapacity];
        this.n = 0;
    }

    …
```

- Referensen till den interna arrayen deklareras med typparametern, men …
- … array-objektet måste skapas som new Object[]
  - Vi kan inte skapa ett objekt av okänd typ …

En generisk list-klass

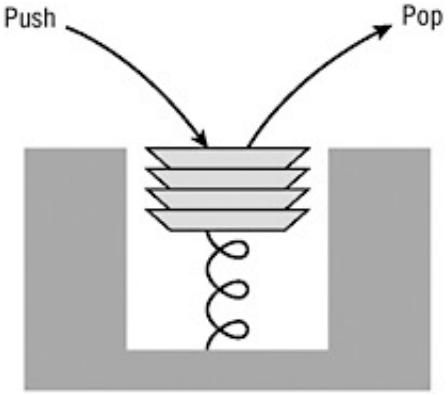
```
public class GenericList<E> {
      …

    public E get(int index) {
        checkIndex(index);
        return elements[index];
    }

    public void add(E obj) {
        if (n >= elements.length) expand();
        elements[n] = obj;
        n++;
    }
```

- Att den interna arrayen är av typ Object[] syns dock inte utanför klassen

Försök själv: En generisk Stack



- Klassen har endast ha två metoder för att modifiera stacken
  - push – nytt objekt placeras på toppen
  - pop – tar bort, och returnerar objektet som ligger på toppen
- Andra metoder
  - size – antalet objekt i stacken
  - isEmpty
  - peek – returnera en referens till objektet på toppen utan att ta bort det
- Privat hjälpmetod, expand, för att utöka den interna arrayen vid behov

Försök själv: En generisk Stack

- Generisk array-referens, T[] men …
- … själva arrayen måste internt vara av typen Object[]

```
public class GenericStack<E> {

    private E[] elements;
    private int n;

    public GenericStack(int initialCapacity) {
        this.elements = (E[]) new Object[initialCapacity];
        this.n = 0;
    }
```

java.util.ArrayList

- En generisk listklass finns redan implementerad i Javas API
  - Enklare att använda än en array om element läggs till och/eller tas bort ur samlingen
- Hjälpmetoder i java.util.Collections (klassmetoder, static)
  - Collections.sort(list)
  - Collections.shuffle(list)
  - …

java.util.ArrayList

```
ArrayList<String> band;
band = new ArrayList<String>();

String vocals =
   new String("Robert Plant");
String guitar =
   new String("Jimmy Page");
String bass   =
   new String("John Paul Jones");
String drums  =
   new String("John Bonham");

band.add(vocals);
band.add(guitar);
band.add(bass);
band.add(0, drums);
```

```
int n = band.size();

String artist = band.get(3); // JPJ

artist = band.remove(0); //JB

for(String s : band) {
    System.out.println(s);
}

// Sad, but true
band.clear();
```